2025 Undergraduate Hypersonic Flight Design Competition

University Consortium for Applied Hypersonics

University of Notre Dame

December 10, 2025

Authors: Sebastian Viegas, Zachary Gurland, Christopher Qian, Joseph Zinkan, and Benjamin Creamer

Executive Summary

This report presents the design, analysis, optimization, and experimental validation of an unpowered, low-altitude hypersonic projectile developed for the 2025 Undergraduate Hypersonic Flight Design Competition. The mission objective is to create a vehicle capable of carrying a cylindrical payload while remaining within strict dimensional, mass, aerodynamic, thermal, and financial constraints. The projectile must fly between Mach 5 and 8, remain below 9 km in altitude, demonstrate or achieve a 15 g maneuver, and impact the ground at no less than Mach 2, all while maintaining structural integrity and internal temperatures below 74°C.

The design effort began with an extensive review of historical and contemporary hypersonic configurations. Early trade studies considered delta shapes, flat-bottom bodies, and flared cones, drawing heavily from Becker, Zhao, Jin, Pan, and others. Initial CFD-tested iterations of a blunted-flared cone were unable to produce a sufficient lift-to-drag ratio, achieving no more than L/D = 1.172 at a 4° angle of attack. This deficiency motivated a shift toward waverider-based geometries. Guided by the osculating-cone methodology and supported by MATLAB-generated geometric construction, the team pursued a blunted, osculating-cone-derived waverider configuration. This approach preserved high lift-to-drag performance at Mach 6 while also providing robustness against intense heating loads inherent to low-altitude hypersonic flight.

To meet manufacturability and payload integration requirements, the final design incorporates a flared cylindrical payload compartment on the upper surface. This feature was carefully aligned with the lower-surface slope to minimize parasitic drag and preserve attached flow. Additional shaping modifications, including 5 mm leading-edge fillets, ensured compliance with the dimensional limits without sacrificing aerodynamic coherence. The final vehicle length is 0.542875 m, with a maximum radius of 0.097 m, meeting the competition's strict dimensional constraints. Mass properties were also brought into compliance, with the completed design weighing 22.143 kg and achieving a high volume efficiency of 0.67566.

Performance analysis centered on ANSYS Fluent predictions at Mach 6. With an angle of attack of 4°, the projectile body produces 3794 N of lift, exceeding the 3384.45 N minimum needed to execute a 15 g maneuver. With fin deflections of –5°, the projectile exhibits a net positive pitching moment at this condition, verifying that less trim authority is required and consequently affirming the ability to maneuver at or above 15 g. At higher angles of attack, lift increases further, reaching 5174 N at 11°, providing additional margin. Using the calculated turn radius of 22.5 km and an initial velocity of 1818 m/s, trajectory and energy-balance analysis show that the projectile impacts the ground at Mach 2.1, above the Mach 2 requirement, with a descent angle of 53.2° and a ground-referenced range of 20,850 km. Using L/D = 3.1 at a 6° angle of attack, the theoretical maximum range was approximated at 120 km using the energy-height method.

Thermal protection and structural integrity were major drivers of the design. The structural frame is constructed of 2 mm Ti-6Al-4V, selected for its combination of strength, stiffness, fatigue resistance, and high-temperature performance. The thermal protection system uses 3 mm reinforced carbon-carbon (RCC) coating supported by 5 mm of silica aerogel insulation. CFD-derived heating maps identified concentration of thermal loads near the bow and at geometric transitions between the conical and curved surfaces; insulation was applied accordingly. To ensure static stability, 9.334 kg of tungsten ballast was precisely placed in the nose and forebody compartments, shifting the center of mass ahead of the center of

pressure throughout flight. This guarantees positive static margin, consistent with the center of mass measurement of 365.25 mm from the leading edge.

Internally, avionics – including the flight computer, IMU, battery pack, and actuators – were integrated using custom 3D-modeled housings positioned between titanium bulkheads. This configuration increases stiffness, prevents deformation under acceleration and aerodynamic loading, and preserves internal volume for both insulation and payload. Two all-moving fins arranged in an inverted-V configuration (110° included angle) provide control in pitch, yaw, and roll, meeting maneuver and trim requirements while minimizing drag at hypersonic speeds.

A full suite of computational and experimental tools supported this design process. SolidWorks provided watertight geometry for CFD and enabled precise mass-property estimates. ANSYS Workbench delivered aerodynamic pressure fields, convective heat flux, wall temperature, and shear stress data. MATLAB supported waverider hull generation and post-processing of experimental thermal imagery. Wind-tunnel validation was conducted in Notre Dame's Mach 6 Ludweig Tunnel using a 0.5-scale Rigid-10K model. Schlieren imaging verified major shock structures and expansion regions, while infrared thermography captured surface heating trends. Experimental heating distributions validated most CFD-predicted patterns, particularly bow-edge heating and surface sweeps across upper geometries. Discrepancies on the lower surface, likely caused by the roughened underside of the model, did not affect confidence in the overall thermal trends used to size the TPS.

A detailed breakdown of materials, masses, and costs confirms that the final design meets resource requirements while remaining manufacturable. The thermal protection system weighs 1.044 kg and costs $1,250; the titanium frame weighs 1.540 kg; tungsten ballast accounts for 9.334 kg at $513.37; avionics contribute approximately 0.225 kg; and actuators add 0.260 kg. Total component mass excluding payload is 13.143 kg, with an estimated cost of $4,188.65.

Overall, the final waverider projectile meets all mission requirements. It fits within dimensional limits, meets mass and volume efficiency constraints, maintains static stability, operates at Mach 6, can execute 15 g maneuvers, and impacts at Mach 2.1. While a full transient thermal analysis requires additional simulation work, the design presently incorporates appropriate historical thermal-protection approaches, insulation layers, and reserved space for phase-change materials. The combined computational and experimental results validate the aerodynamic, thermal, and structural viability of the design, establishing a strong foundation for fabrication, testing, and launch-readiness.

Acknowledgements

We would like to extend our special thanks to Alec Jobbins and Elio Carella for offering their support and expertise in running and analysing computational fluid dynamics simulations and with troubleshooting. We also thank Jacob Caldwell and Alec Jobbins for their invaluable assistance with the wind tunnel experiment setup and data acquisition, as well as Alysa Spencer for dedicating her time to support the experimental testing. Finally, we express our sincere thanks to Professor Thomas Corke for organizing the design team and providing essential guidance, mentorship, and support throughout the research and design process.

Table of Contents

a. Executive Summary ……..i
b. Acknowledgements ……..……iii
c. Table of Contents ……iii
d. Nomenclature ……..iii
I. Requirements Definition ……..1
II. Concept of Operation ……...1
III. Trade Studies ……2
IV. Design and Fabrication…….5
V. Methods and Tools ……...8
VI. Mission Summary ……..10
VII. Experimentation and Plan for Launch ……...12
Appendices
A. Codes ……..13
B. ANSYS Notes ……22
C. ANSYS Fluent Results …….. 22
Bibliography ……24
CAD Drawings ……25

Nomenclature

C - Celcius
CAD - Computer-aided design
CFD - Computational Fluid Dynamics
CoM - Center of mass
CoP - Center of pressure
IMU - Inertial measurement unit
M - Mach number
L/D - Lift-to-drag ratio
RCC - Reinforced carbon-carbon
Re - Reynold's number

I. Requirements Definition

The mission is to design and analyze an optimized unpowered, low-altitude, hypersonic projectile that meets a set of requirements. Specifically, the vehicle must carry a cylindrical payload and remain within strict dimensional and mass limits. Over its mission, the vehicle is launched from the ground, traverses the atmosphere in a high-speed trajectory within bounds Mach 5-8, executes or demonstrates the capability for a 15g maneuver at a maximum of 9 km in altitude, and impacts the ground at a speed of at least Mach 2 at sea level, all while maintaining structural integrity and internal thermal limits.

The vehicle requirements can be categorized into dimensional, aerodynamic, thermal, and financial constraints. Dimensionally, the vehicle has a maximum length of 1 meter, a maximum diameter of 0.2 meters, a maximum mass of 45 kg, and a minimum volume efficiency of 0.12. Aerodynamically, the vehicle must fly between M 5 and 8, must demonstrate trim to execute a maneuver, must be statically stable, must operate at or below 9 km altitude, must execute a 15g or more maneuver, and must impact the ground at M 2 or greater. Thermally, all components must survive thermal loading during flight and must maintain an internal temperature of below 74 degrees Celsius (C) at impact. Financially, materials cost, assembly cost, and fabrication costs must be kept to a minimum. These form the basis for the top-level design criteria for the vehicle.

II. Concept of Operation

The proposed design is a modified osculating-cone, lift-generating hypersonic projectile. The design was created with the method of multiple osculating cones. It is compliant with maximum length and diameter limits, accommodates the required payload mass, and is oriented around Mach 6, while preserving stability and trim capability.

In order to successfully complete a 15g or higher maneuver, the projectile must be able to produce at least 3384.45N of lift with its body based on its mass and the acceleration due to gravity. The control surfaces must then be able to deflect such that there is no pitching moment at the angle of attack that produces this lift. At an angle of attack of 4 degrees, the projectile body produces 3794N of lift force according to ANSYS Fluent results. At a fin deflection angle of -5 degrees, the projectile has a net positive pitching moment at this angle of attack, meaning with less trim the projectile can complete a maneuver at above 15g.

To determine the speed at which the projectile will impact the ground, various calculations must be done. With an initial velocity of 1818m/s (M 6 at 9 km altitude), generated lift of 3794N, and a weight of 22.143 kg, this means there is enough lift to make a 15.5g turn. Assuming the turn is made at only 15g, the turn radius will be about 22.5 km. Using geometry, the impact angle will be 53.2 degrees with a distance traveled of 20.850 km. Using energy balance to determine the final velocity, the impact speed will be M 2.1 at sea level – higher than the M 2 minimum. It is relevant to note that as the angle of attack increases to 11 degrees maximum lift generated reaches 5174N, much more than what is necessary for a 15g turn.

In terms of maximum theoretical range, this can be approximated using the energy height method. At the projectile's ideal L/D of 3.1 at an angle of attack of 6 degrees, using the data from Appendix C and the energy height method shows that the maximum theoretical range of the projectile will be about 120 km.

With regard to each constraint listed in Section I, the proposed design has a length of 0.542875m, fits within the maximum diameter with a maximum radius of 0.097m, has a mass of 22.143 kg, has a volume efficiency of 0.67566, operates at M 6, is stable in pitch, operates at a maximum altitude of 9 km, and is optimized to be as affordable and manufacturable as possible. In terms of the thermal constraints, exact data was unable to be calculated. In order to do so, an ANSYS Fluent simulation at M 6 and 9 km would need to be run to get a reference temperature field and heat transfer coefficient. This would then be run through an ANSYS Mechanical transient thermal simulation with the materials stack inputted. An MATLAB code would then need to be created to generate a velocity vs. time profile that can be ported into the simulation. This ANSYS Mechanical simulation would return the internal temperature of the projectile along its trajectory. In lieu of this, the projectile has been designed with layers of material based on historical designs, and space has been left for phase change material to be wrapped around the internal components in the event the aerogel insulation is not insulative enough.

It is statically stable in pitch because the center of gravity is closer to the bow than the center of pressure as can be seen in Figure 1 below. The yellow sphere represents the position of the center of pressure. This visualization depicts the center of pressure at a 10 degree angle of attack but it was found that it changes only slightly forward at lesser angles of attack. The coordinates of the point depicted in the graph have a nonzero z value but in reality the z value would be 0 and along the plane of symmetry.

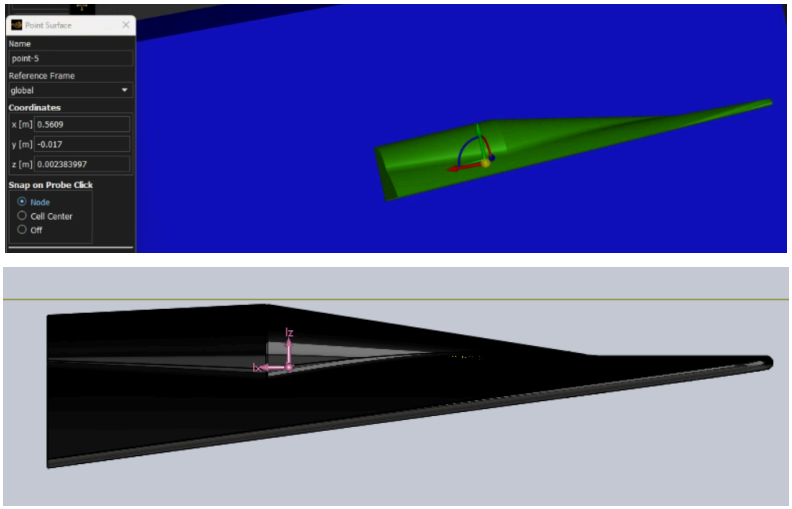


Figure 1. (a) *top* Center of Pressure Location (b) *bottom* Center of Mass estimate taken from leading edge of flight vehicle (CoM = 365.25 mm)

III. Trade Studies

Conceptual design for the proposed hypersonic projectile began with considering high lift-generating bodies. Initial research into this body design was done with Becker. Becker considered M ranges from 5-20, but more importantly for the final design of the projectile delta, flat bottom, half cone, and flat top projectile shapes are considered in terms of volumetric efficiency and L/D. From this paper, flat bottom shapes were determined to be favorable due to their high volume efficiency relative to their L/D [1].

Next, consideration was given to waveriders that could operate efficiently at multiple M numbers. Waveriders were explored due to their relatively high L/D when operating at hypersonic speeds. In this

vein, Zhao et al. and Jin et al. were consulted for their work in design methods and analysis of wide-range hypersonic glide vehicles. They discussed methods such as expanding the side profile to adjust to M changes, design the body for different M as you move outwards spanwise, and even combining two bodies designed for different M [2,3]. After some consideration in terms of the feasibility of designing these body shapes ourselves, it was determined that creating and adapting these specialized and complex body shapes would likely be out of the scope of the team's skillsets and so other avenues were pursued.

Based on the historical designs of hypersonic projectiles, it was decided that a blunted-flared cone design would be optimal for its stability, volume, and ease of manufacturing. After several iterations, a final design for this cone was configured as shown in Figure 2.

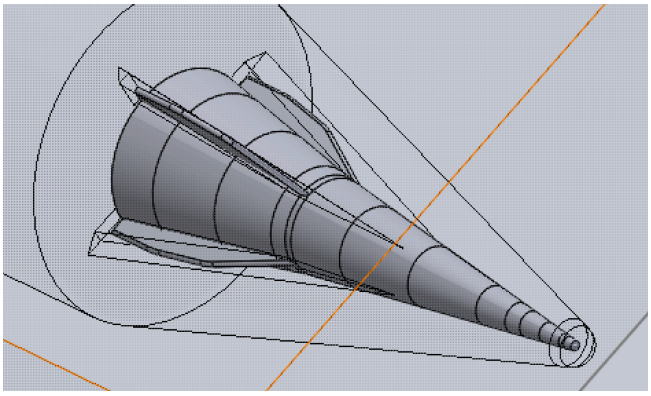
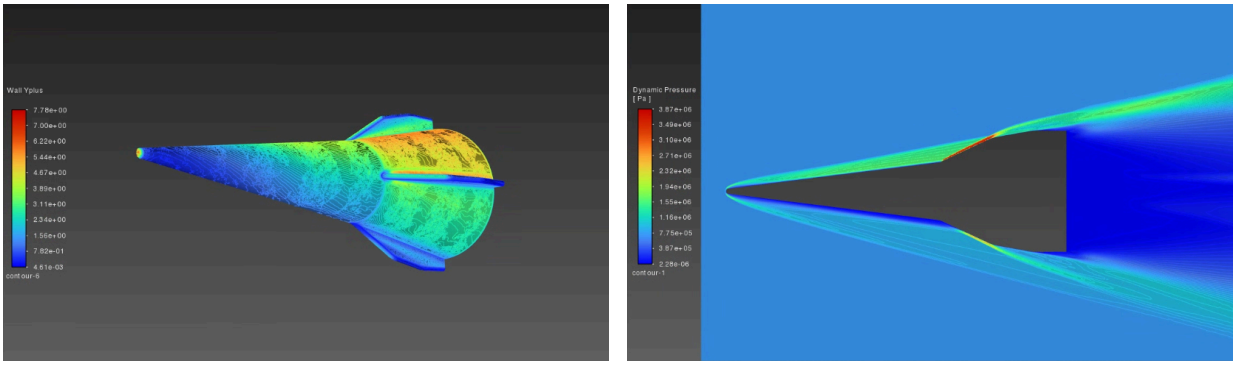
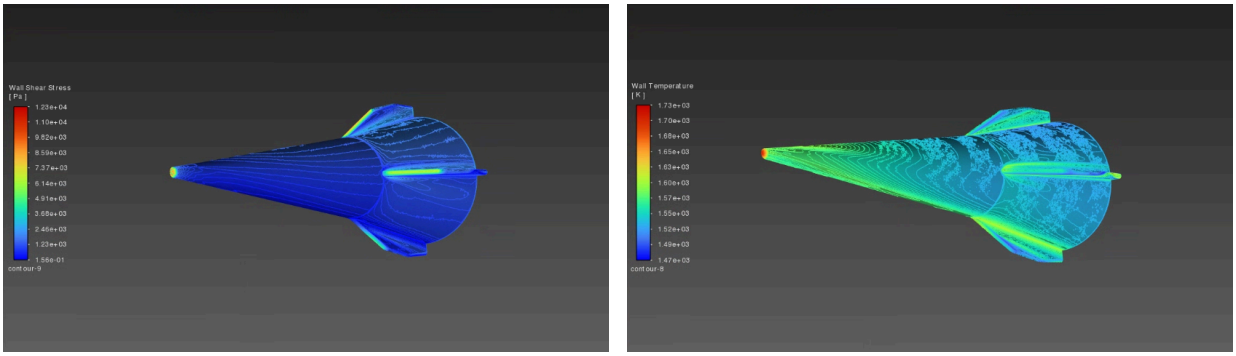


Figure 2. Final iteration of the blunted-flared cone projectile with wireframe bodies of influence in frame

After CFD testing in ANSYS Fluent of this design, it was determined that this shape was unable to produce the desired L/D and would have to be changed. After multiple iterations and CFD runs in an attempt to optimize the L/D ratio, the team could not surpass a L/D ratio of 1.172 at a 4 degree angle of attack (the industry standard assumption for max L/D) for this particular projectile geometry. The results concerning wall temperature, wall shear stress, dynamic pressure and wallyplus, all which asserted the inadequacy of the blunted-flared cone projectile design, are depicted in Figure 3 numbered a-d



(a) **(b)**



(c) **(d)**

Figure 3. CFD analysis results for the blunted-flared cone projectile at 4 degree angle of attack depicting the (a) Wall Yplus (b) Dynamic Pressure (c) Wall Shear Stress and (d) Wall Temperature

Once the team had decided that this L/D ratio was insufficient, further research was conducted once again into possible lifting body designs. From Pan, et al., it was determined that the flared cone design was favorable for stability but not optimal for maximizing L/D [4]. Furthermore, Gloria determined that including a flared feature on only a relatively small portion of the body did not contribute to a significant decrease in L/D [5]. As a result, a decision was made to incorporate a flared extrusion on the upper surface of the final design that would allow for more internal volume while having as little effect on the overall L/D as possible.

In terms of the general design for the body shape, it was determined from Scott that an osculating cone derived waverider design optimized for M 6 would be ideal [6]. This design can be parameterized for design constraints and payload using computer coding. Furthermore, a blunted waverider design was chosen due to the extreme heating that the body would inevitably face at low altitudes. Work by Guo, et al. showed that this adjustment can be made to a waverider body without harming L/D catastrophically and the design of the waverider can even be optimized for having blunted edges [7].

In the end, a blunted, osculating cone-derived, hypersonic waverider design with a flared cone on the upper surface for housing payload was chosen. Based on research, this design was determined to most effectively balance the many constraints and requirements of the projectile design while remaining feasible to produce. A SolidWorks based engineering drawing of the final design it depicted below in Figure 4. Furthermore, the physical model of the final design, printed in Rigid 10k resin at half scale for wind tunnel testing, can be seen attached to a sting in the Mach 6 wind tunnel in Figure 5.

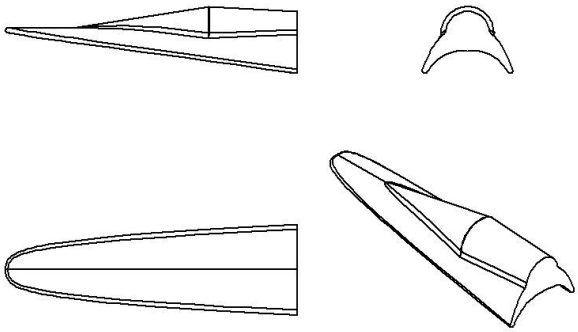


Figure 4. Engineering drawings for geometry of the gun test article waverider body design

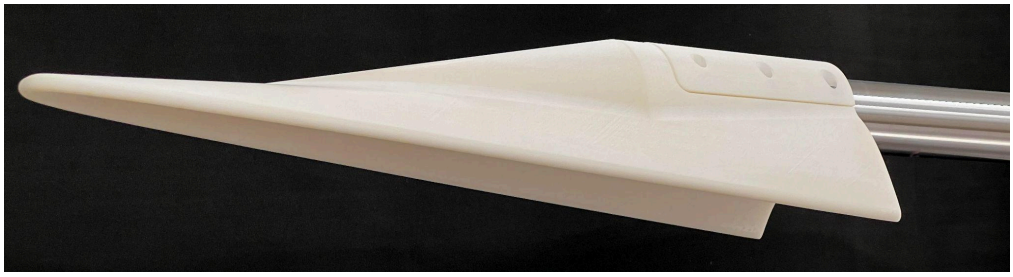


Figure 5. Waverider Projectile 0.5 Scale Model Attached to Sting

IV. Design and Fabrication

The overall projectile design was initially generated using MATLAB-based geometric synthesis scripts, described in detail in Section V. These routines produced a baseline body shape that exceeded the allowable dimensional envelope. To resolve this, the leading edges were fileted to a 5 mm radius, which reduced the projected span and allowed the geometry to satisfy the dimensional constraints while maintaining acceptable aerodynamic continuity. This modification, however, significantly reduced the available internal volume within the main body. As a result, a large cylindrical payload compartment was added along the upper surface. Its longitudinal axis was tilted to match the slope of the lower surface, a configuration chosen specifically to minimize its parasitic drag contribution and to preserve attached flow over the aft body.

To further streamline the integration of this compartment, a downsloping conical forebody extension was introduced ahead of the cylinder. This feature both smooths the external flow transition to mitigate adverse pressure gradients and interference drag, and provides dedicated internal space for the avionics bay and a tungsten ballast mass, strategically positioned to shift the center of mass forward for improved static stability. The final prototype also incorporates two all-moving fins functioning as aerodynamic control surfaces. These fins are arranged in an inverted-V configuration with a 110° included angle, providing effective control authority in pitch, yaw, and roll throughout the flight envelope.

After the external geometry of the hypersonic vehicle had been established, the materials for the thermal protection system, structural frame, and insulation were taken into consideration. For the overall structural frame, the team selected 2 mm of titanium alloy (Ti-6Al-4V Solution treated and aged). This selection was based on the reasoning that titanium alloy provides the best balance of strength, stiffness, and mass for the projectile's extreme loading environment. This alloy offers high yield strength, excellent fatigue resistance, and stability at elevated temperatures, making it capable of withstanding gun-launch acceleration and hypersonic dynamic pressures. At 2 mm thickness, the frame achieves sufficient rigidity and safety margin without adding unnecessary weight, outperforming thinner sections that lacked strength and thicker sections that only increased mass. Next, for the thermal protection system, the team decided to use 3 mm of reinforced carbon-carbon (RCC) coating on the outer surface of the titanium. This decision was based on RCC's exceptionally high temperature tolerance, low thermal conductivity, and proven performance in severe aerodynamic heating environments. Its ability to withstand sharp transient heat loads without structural degradation made it the most reliable option for protecting the titanium frame during hypersonic flight, while the 3 mm thickness provided sufficient thermal margin without excessive added mass. The next critical material selected by the team was 5 mm of silica aerogel. This insulative layer provides extremely low thermal conductivity, allowing it to protect the internal electronics and

payload from the high external surface temperatures experienced during hypersonic flight. The choice of 5 mm thickness creates a strong thermal buffer while adding minimal mass, making it an efficient and reliable choice for preserving internal component functionality under severe heating conditions. Moreover, the silica aerogel was strategically placed based on the location of the major heating loads, wall shear stresses, and dynamic pressure loading as identified during CFD, which are pictured in Figure 6 a-d below

These heating loads, and therefore the corresponding silica aerogel, are concentrated near the tip of the vehicle and around the transition from the curved lower body of the flight vehicle to the conical upper surface.

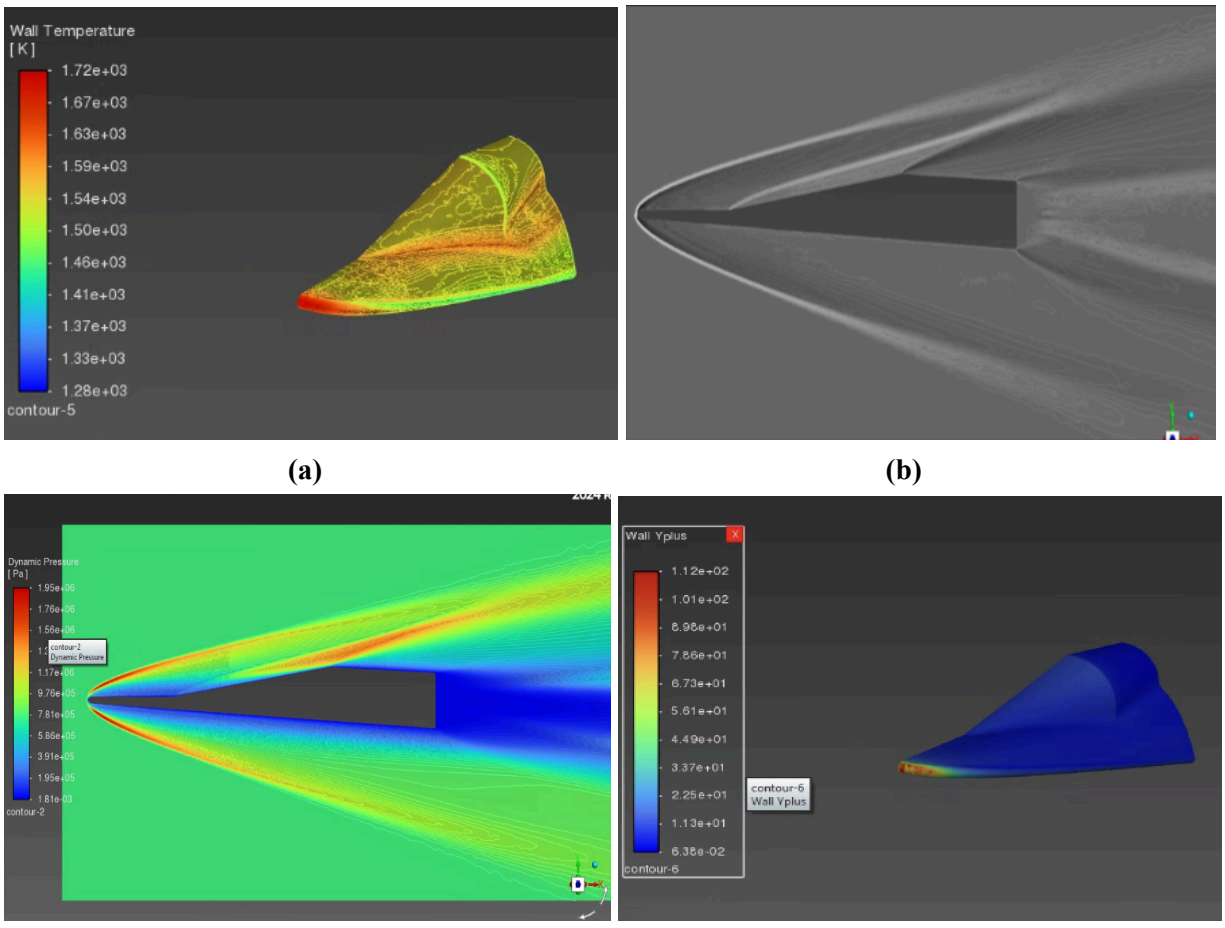


Figure 6. CFD analyses depicting the location of the most intense heating loads (a), Schlieren imaging (b), dynamic pressure loading (c), and wall shear stresses in positive Y-direction (d) used to justify the positioning of the internal silica aerogel insulation

Furthermore, the team made sure to allot reserve space for any necessary phase change material that will further protect the internal components from the extreme heat loads generated by the Mach 6 flight conditions. The final critical material included in the SolidWorks model was the tungsten ballast weight. To ensure static stability in pitch, the center of mass needed to remain forward of the center of pressure during flight. To achieve this, 9.334 kg of tungsten was added to the nose and conical

compartments of the projectile and extruded toward the front of the flight vehicle. Tungsten was selected because its extremely high density of 19,000 kg/m³ allows a large mass shift within a small volume, and its high melting point ensures it can survive the severe thermal environment encountered during hypersonic flight.

In addition to the materials that comprise the body of the flight vehicle, the team considered the internal design of the projectile to ensure structural integrity while allowing for the positioning of tungsten ballast weight, payload, critical avionics, and power and control actuation devices in a manner that would optimize the mass and center of mass of the projectile. First, three titanium bulkheads were placed at strategic locations along the projectile's length to provide internal support and distribute structural loads. These bulkheads increase rigidity, prevent deformation under acceleration and aerodynamic forces, and help maintain the shape and alignment of the internal compartments. Then, 3D molds for the avionics (flight computer, IMU, battery pack, etc.) were uploaded from McMaster Carr and positioned between the bulkheads and as far forward as their individual geometries would allow. Finally, the payload was inserted into the conical compartment that had previously been designed into the flight vehicle to adhere to its geometry. Rear views of the final internal layout of the flight vehicle, showcasing the multi-layered structural body and the central positioning of the payload, can be seen below in Figures 7 and Figure 8 below.

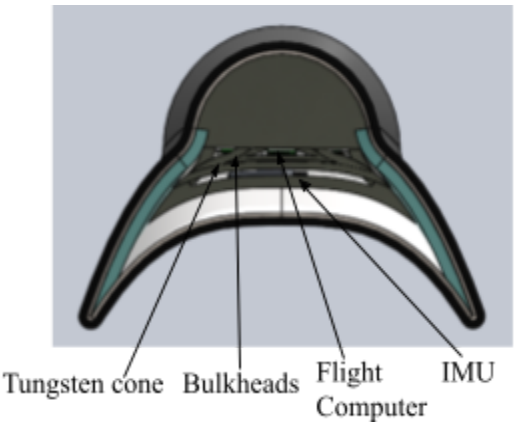


Figure 7. Rear view of internal geometry without the centrally positioned payload

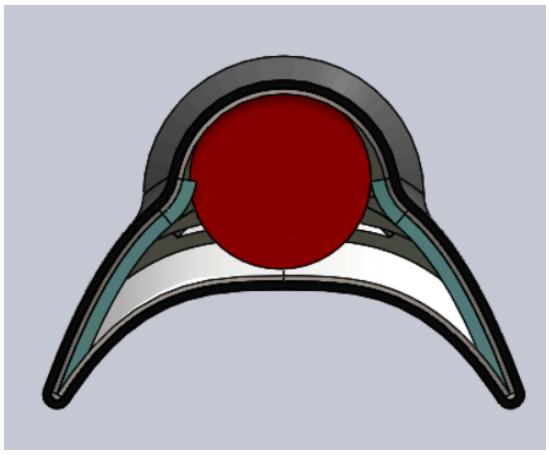


Figure 8. Rear view of the internal geometry of the flight vehicle with the centrally positioned payload

After the design for both the exterior and interior had been established, the team considered the fabrication processes for each of the critical components. The fabrication of the projectile integrates multiple materials and subsystems using methods tailored for manufacturability and performance. The titanium structural frame and bulkheads are produced via stamping and precision machining, ensuring dimensional accuracy and alignment for assembly. The RCC coatings are applied to high-heat surfaces through deposition techniques to provide thermal protection, while silica aerogel insulation and any phase change material are carefully placed within the internal compartments to match CFD-predicted heating patterns. The tungsten ballast weights are machined to shape and bonded into the conical forebody and nose to achieve the desired center of mass. Avionics components are installed between bulkheads in

pre-fabricated 3D housings to simplify assembly and maintain structural clearance. Finally, all layers are assembled with a careful alignment check to ensure that aerodynamic performance, structural integrity, and subsystem integration are preserved throughout the vehicle.

V. Methods and Tools

To create the body shape of the proposed waverider projectile, MATLAB codes were used to generate contour lines that could be ported into SolidWorks. These codes are comprised of a master script titled create_waverider.m along with several helper codes titled *calculate_panel_LD.m*, c*one_angle.m*, *cone_field.m*, *f_cone_root.m*, *shock_angle.m*, *Taylor_Maccoll.m*, and *Vt0.m*. Using these scripts, the final projectile design was created with the following parameters: M 6 speed, shock angle of 12 degrees, height at the base plane of 0.15 meters, half-width at the base plane of 0.105 meters, and an X1,X2,X3,X4 of 0.001,0.001,0,0 respectively. The code also uses the panel method to estimate the L/D of the waverider at an inputted angle of attack, in this case 4 degrees. The estimated L/D of this design was 14.280.

Once the streamlines were ported into SolidWorks, they were used to create a solid body of the waverider and the leading edges were fileted to 5mm radius. This allowed the waverider to fit within the given spatial constraints. The Solidworks model was then put into ANSYS Fluent with Fluent Meshing where it was given a fine mesh and tested at angles of attack ranging from -2 to 11 degrees. These were used to see how much lift the body generated, where the center of pressure was, and find the L/D. SolidWorks was also used to model the materials and interior components of the projectile. With this, a center of mass and a total mass could be calculated. With this mass, the necessary lift to perform a 15g turn could be determined. This lift could be compared with the lift generated by the body to find which angle of attack is needed to initiate the turn. Then, the fins could be deflected until there is a net 0 pitching moment, which means that at that specific trim, the waverider could perform and hold a 15g turn. Figure 9 below shows the workflow process of the design of the proposed projectile.

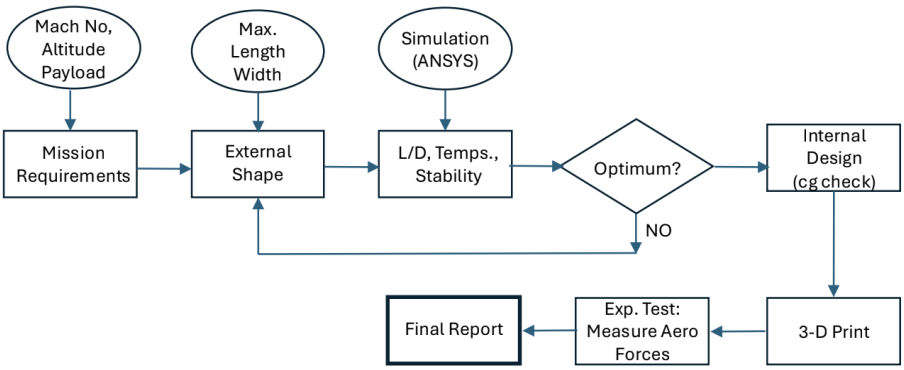


Figure 9. Hypersonic Projectile Design Flow

V – A. Validation

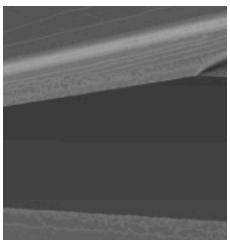
The CFD results from ANSYS Fluent came from utilizing a mesh sizing of a tenth of a millimeter of the wall surfaces of the projectile. This was chosen because it would create a fine mesh that would lead

to increased accuracy in the results. While the exact values from the CFD results could not be validated experimentally with the available tools, the heating trends and shock waves could.

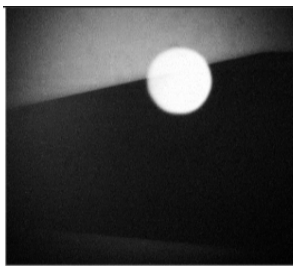
Since the M 6 tunnel available in reality produced a M 5.83 flow, a CFD with the aforementioned mesh sizing was run at a -2 degree angle of attack and M 5.83 flow to mimic the conditions in the tunnel for the scale model. Table 1 shows the calculated results from ANSYS in side-by-side comparison with the experimental results from the wind tunnel. The experimental results on the right hand side of the table were refined using the *FLIRConversion.m* code. The Schlieren imaging was unable to be suitably refined using background deletion and so only a moderately enhanced image is provided.

With regards to the Schlieren imagery, the experimental results confirmed the computed shock cone location of the oblique shock generated by the cone-shaped extrusion on the upper surface. This can be seen when comparing Table 1.A with 1.B, although the Schlieren lines are difficult to see in 1.B due to an inability to properly enhance the image. When looking at the infrared thermography of the projectile model, there is generally consistency across the upper surface between the computed and experimental results. The computed results indicated areas of high heating at the bow leading edge that sweeps down as you move aft along the leading edge. It also computed high heating at the intersection of the cone and cylinder extrusions and the rest of the upper surface along with an angled sweep of heating above the intersection on the side of the cylinder extrusion. There was also a band of low heat on the upper edge between the cone and cylinder extrusion. These results can be seen in Table 1.C and E. Table 1.D confirms the trend of high heat along the edge of the cylinder and upper surface as well as the sweep of heating on the side of the cylinder. Table 1.F confirms the trend of high heat at the bow's leading edge that trails into lower relative heat as you move aft along the leading edge as well as the trend of increased heating at the intersection of the cone and upper surface. However, the computed trend of high heat in the center of the lower surface as seen in Table 1.G and I could not be validated experimentally. As seen in Table 1.H and J, the lower surface had increased heat away from the centerline of the projectile and a band of low heat along the centerline. An exact explanation for this discrepancy could not be determined, although it is theorized that the rough-to-the-touch underside of the projectile could be influencing the dynamics of the airflow and causing it to behave undesirably.

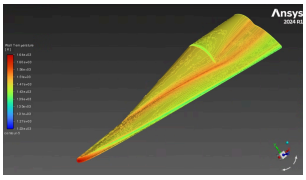
Table 1. Comparison of Computed Data at M 5.83 Against Experimental Data at M 5.83



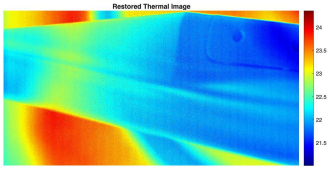
3.A: Logarithmic Streamwise Schlieren Computation



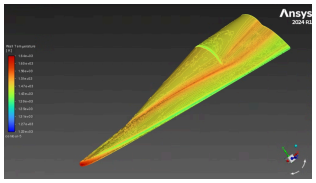
3.B: Experimental Schlieren Imagery



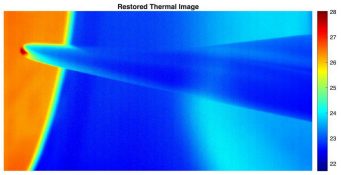
3.C: Heating Trends on the Upper Surface of the Projectile



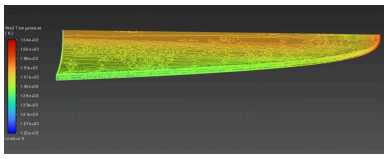
3.D: Heating Trends of the Aft of the Projectile



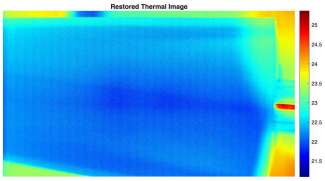
3.E: Heating Trends on the Upper Surface of the Projectile



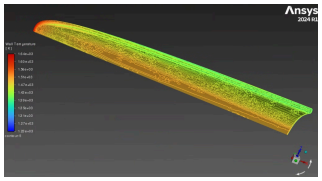
3.F: Heating Trends on the Bow of the Projectile



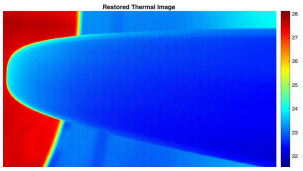
3.G: Heating Trends on the Lower Surface of the Projectile



3.H: Heating Trends on the Underside of the Aft of the Projectile



3.I: Heating Trends on the Lower Surface of the Projectile



3.J: Heating Trends on the Underside of the Bow of the Projectile

VI. Mission Summary

This section consolidates all facility, software, computational, material, and cost resources required to execute the projectile design, analysis, and testing campaign. It provides a structured overview of the tools used to generate aerodynamic, thermal, structural, and trajectory predictions, as well as the

physical resources and materials necessary for fabrication and testing. By presenting both computational workflows and component-level mass and cost estimates, this summary establishes a clear traceability path from design requirements to engineering decisions.

Table 2 below outlines the full suite of analytical and experimental tools used throughout the design process. These resources form the backbone of the engineering workflow, supporting geometry development, aerodynamic characterization, thermal and structural load modeling, and trajectory evaluation. Each tool plays a distinct role in ensuring that performance predictions are grounded in both high-fidelity simulation and experimental verification. The table summarizes each resource, its primary purpose, and key notes relating to how it supports mission-critical analyses.

Table 2. Facility, Software, and Computational Resource Summary

Category	**Resource Requirement**	**Notes**
CAD Tools	SolidWorks	Used to create watertight external geometries for CFD meshing and to generate clean STEP/STL exports for wind tunnel model fabrication. Also supports internal layout, mass property estimation, and CG/CM tuning for stability analyses.
CFD	ANSYS Workbench	Used to compute static and dynamic pressure distributions, convective heat flux, wall temperature, wall shear stress, and turbulent kinetic energy. CFD results feed directly into TPS sizing, fin hinge-moment calculations, and structural load derivation. Geometry iterations are evaluated to improve L/D and reduce heating.
Structural Analysis	ANSYS Workbench and SolidWorks Simulation	Used to quantify bending stress, shear stress, and combined thermo-mechanical loads. Evaluates survivability under gun-launch acceleration and hypersonic dynamic pressure loads. Structural margins are checked for a 15-g maneuver at 9 km using CFD-derived pressure fields.
Wave Rider Geometry Generation and Image Processing	MATLAB	Used to iterate on wave rider hull geometry. Provided L/D estimates and sizing estimates that allowed for optimization of body geometry. Allowed for conversion of FLIR camera data into usable images for comparison and validation of computational results.
Wind Tunnel Testing	Notre Dame Hessert Aerospace & Mechanical Engineering Laboratory Mach 6 Ludweig Wind Tunnel	Used to validate CFD predictions. Schlieren imaging provides visualization of shock structures, expansion fans, and boundary-layer behavior. Infrared thermography provides surface temperature measurements for TPS verification. Used to refine aerodynamic coefficients and confirm shock-on-lip and control surface flow behavior.

Shifting to a more detailed look at the resource requirement of the flight vehicle itself, Table 3 below provides a detailed breakdown of the materials, mass contributions, and cost estimates associated with all major projectile components. These values reflect the current design iteration, incorporating realistic material densities, commercial pricing, and subsystem-level mass allocations. The table highlights the relative cost and weight drivers and establishes a baseline for future optimization, trade studies, and manufacturability assessments.

Table 3. Material, Mass, and Cost Resource Summary for Projectile Components

Component	Material Type	Estimated Weight	Estimated Cost
Thermal Protection System	Reinforced Carbon-Carbon	1.044 kg	$1,250
Structural Frame	Ti-6Al-4V Solution treated and aged (SS)	1.540 kg	$107.80
Insulation	Silica Aerogel	0.0413 kg	$2.48
Ballast Weight	Tungsten	9.334 kg	$513.37
Fins (2)	Reinforced Carbon-Carbon and Ti-6Al-4V Solution treated and aged (SS)	0.029 kg	$10.00
Flight Computer / Autopilot	PCB, connectors, aluminum enclosure	0.065 kg	$450
IMU	MEMS sensor, PCB, aluminum enclosure	0.120 kg	$1,200
Battery Pack	Li-ion pouch cells, polymer wraps	0.600 kg	$120
Wiring + Connectors	Copper wires, Teflon or PVC insulation, metal plugs	0.070 kg	$25
Power Distribution/Regulator	PCB, MOSFETs, aluminum heat sink	0.040 kg	$30
High-torque Actuators	Titanium/steel gears, aluminum case, plastic housing	0.065 kg x 4 = 0.260 kg	$120 x 4 = $480
Total:		13.143 kg (without 9 kg payload)	$4,188.65

VII. Experimentation and Plan for Launch and Wind Tunnel

In order to validate CFD results and calculations, a 0.5 scale model of the final gun test article was 3D printed using Rigid 10K. The scale model was positioned at a -2 degree angle of attack. The goal

of the wind tunnel test was to verify ANSYS Fluent CFD results using Schlieren and infrared imaging and comparing the trends in the computed and experimental data. All wind tunnel tests were conducted in a Mach 6 Ludweig tunnel with a stagnation pressure of 194 psi, a stagnation temperature of 400 degrees Fahrenheit, M 5.83 flow, and a unit Reynolds number of 11.99e6 1/m with an uncertainty of +/- 0.18e6 1/m.

To capture the Schlieren, self-aligned focusing imagery was used to capture Schlieren at the symmetry plane of the projectile body. The imagery apparatus can be seen in Figure 4. The apparatus was focused on the upper surface of the projectile at about the halfway point along the projectile body. Ideally, the leading edge of the bow would be captured, but the viewing portal of the wind tunnel did not allow for this. This resulted in very faint image capture of the Schlieren, but at 200 psi the Schlieren lines became discernable. The raw image results contain a large white circle, which is glare from the light source of the set up.

To capture infrared imaging of the projectile at M 6 flow, a FLIR X8581 High Speed camera was used. This camera recorded at 325 frames per second to capture the heating on the surface of the projectile during the approximately 50 milliseconds of steady flow in the chamber. Rigid 10k was chosen as the build material of the projectile which is semi-translucent to infrared thermography. A translucency correction has not been applied to the data and thus the raw temperature data only describes trends and not quantitative data about the heating characteristics of the projectile.

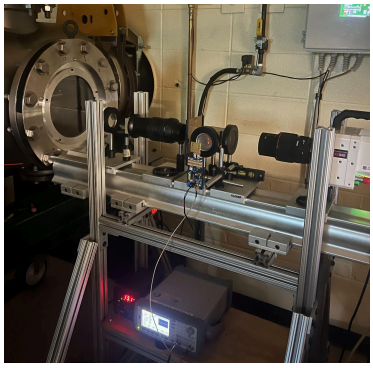


Figure 10. Self-aligned focusing imagery apparatus

The bottom side of the waverider has been sanded to 800 grit since the 3D printing supports were on the bottom side and needed to be removed, which increases the reflectivity of Rigid 10K and reduces the signal that the Mid Wave IR (MWIR) camera is capable of capturing. Therefore, the temperature changes on the lower surface of the waverider is likely higher than the data indicates. Therefore, while the data shows trends on the top and bottom surfaces individually, the specific trend of high heat on the lower surface relative to the upper cannot be concluded from the data.

APPENDICES

A. Codes

- create_waverider.m

```
- %==========================================
  ============================
- % create_waverider.m
- % This script is a MATLAB translation of the
  'waverider_generator' Python
- % package, originally created by Jade Nassif.
- % It generates a hypersonic waverider
  geometry based on the osculating
- % cones method using a set of 4 design
  parameters.
- % This single file contains the main script
  and all necessary helper
```

```
% functions.
% REQUIRES: The 6 flowfield functions saved in the same directory:
%   - Taylor_Maccoll.m
%   - Vt0.m
%   - cone_angle.m
%   - cone_field.m
%   - f_cone_root.m
%   - shock_angle.m
%=========================================================================
function create_waverider()
    clc;
    clear;
    close all;
    fprintf('Starting waverider generation...\n');

    %=========================================================================
    % --- 1. SET INPUT PARAMETERS ---

    %=========================================================================
    useFlatBottom = true;

    aoa_deg = 5.0;
    M_inf = 6.0;         % Freestream Mach number
    beta_deg = 15;     % Shock angle (degrees)
    height = 0.17;        % Height of waverider at base plane
    width = 0.1;         % Half-width of waverider at base plane
    dp = [0.5, 0.0005, 0, 0]; % 4 design parameters [X1, X2, X3, X4]

    % --- Interpolation & Meshing Controls ---
    n_upper_surface = 100; % Points for upper surface interpolation
    n_shockwave = 100;     % Points for shockwave interpolation
    n_planes = 20;          % Number of osculating planes (>= 10)
    n_streamwise = 20;      % Number of streamwise points per streamline (>= 10)
    delta_streamwise = 0.05; % Max step size for streamline ODE (as fraction of length)
    gamma = 1.4;             % Ratio of specific heats


    %=========================================================================
    % --- 2. INPUT VALIDATION ---
    %=========================================================================
    if ~isnumeric(M_inf) || M_inf <= 0
        error('Mach number must be a positive number');
    end
    if ~isnumeric(beta_deg) || beta_deg <= 0 || beta_deg >= 90
        error('beta must be a number between 0 and 90 degrees.');
    end
    if ~isnumeric(height) || height <= 0
        error('height must be a positive number');
    end
    if ~isnumeric(width) || width <= 0
        error('width must be a positive number');
    end
    if ~isvector(dp) || numel(dp) ~= 4
        error('dp must be a list containing 4 elements');
    end
    X1 = dp(1);
    X2 = dp(2);
    X3 = dp(3);
    X4 = dp(4);
    if ~((X1 < 1 && X1 >= 0) && (X2 <= 1 && X2 >= 0))
        error('X1 and/or X2 are not in the required range');
    end
    if ~((X2 / ((1-X1)^4)) < (7/64) * (width/height)^4)
        error('Condition for inverse design not respected, check X1 and X2');
    end
    if X3 < 0 || X3 > 1
        error('X3 must be between 0 and 1');
    end
    if X4 < 0 || X4 > 1
        error('X4 must be between 0 and 1');
    end
    if n_upper_surface < 10 || n_shockwave < 10 || n_planes < 10 || n_streamwise < 10
        error('n_upper_surface, n_shockwave, n_planes, and n_streamwise must be >= 10');
    end

    %=========================================================================
    % --- 3. INITIAL GEOMETRY SETUP ---

    %=========================================================================
    fprintf('Setting up initial geometry...\n');

    % --- Compute Deflection Angle (theta) ---
    beta_rad = beta_deg * pi / 180;
    tanTheta = 2 * cot(beta_rad) * (M_inf^2 * sin(beta_rad)^2 - 1) / (M_inf^2 * (gamma + cos(2*beta_rad)) + 2);
    theta_rad = atan(tanTheta);
    theta_deg = theta_rad * 180 / pi;

    % --- Vehicle Length ---
    length = height / tan(beta_rad);

    % --- Shockwave Control Points (5 points) [z_bar, y_bar] ---
    % Note: Python is 0-indexed, MATLAB is 1-indexed.
    s_cp = zeros(5, 2);
    s_cp(:, 1) = linspace(X1 * width, width, 5)'; % z_bar coords
    s_cp(5, 2) = X2 * height;
% y_bar coord of last point

    s_P0 = s_cp(1, :);
    s_P1 = s_cp(2, :);
    s_P2 = s_cp(3, :);
    s_P3 = s_cp(4, :);
    s_P4 = s_cp(5, :);

    % --- Upper Surface Control Points (4 points) [z_bar, y_bar] ---
    us_cp = zeros(4, 2);
    us_cp(:, 1) = linspace(0, width, 4)'; % z_bar coords
    us_cp(1, 2) = height;
    us_cp(2, 2) = height - (1 - X2) * X3 * height;
    us_cp(3, 2) = height - (1 - X2) * X4 * height;
    us_cp(4, :) = s_P4; % Last point is shared

    us_P0 = us_cp(1, :);
    us_P1 = us_cp(2, :);
    us_P2 = us_cp(3, :);
    us_P3 = us_cp(4, :);
```

```
-
-       %=========================================
       ==========================
-           % --- 4. CREATE GEOMETRIC INTERPOLANTS
       ---
-
       %=========================================
       ==========================
-
-           % --- Create Upper Surface Interpolant
       ---
-           % (Was
       self.Create_Interpolated_Upper_Surface)
-           t_values_us = linspace(0, 1,
       n_upper_surface);
-           points_us = zeros(n_upper_surface, 2);
-           for i = 1:n_upper_surface
-               points_us(i, :) =
       Bezier_Upper_Surface(t_values_us(i), us_P0,
       us_P1, us_P2, us_P3);
-           end
-           % Create griddedInterpolant (MATLAB's
       equivalent of interp1d object)
-           Interpolate_Upper_Surface =
       griddedInterpolant(points_us(:, 1),
       points_us(:, 2), 'linear');
-
-           % --- Create Shockwave Interpolant ---
-           % (Was
       self.Create_Interpolated_Shockwave)
-           t_values_sw = linspace(0, 1,
       n_shockwave);
-           points_sw = zeros(n_shockwave, 2);
-           for i = 1:n_shockwave
-               points_sw(i, :) =
       Bezier_Shockwave(t_values_sw(i), s_P0, s_P1,
       s_P2, s_P3, s_P4);
-           end
-           Interpolate_Shockwave =
       griddedInterpolant(points_sw(:, 1),
       points_sw(:, 2), 'linear');
-
       %=========================================
       ==========================
-           % --- 5. FIND OSCULATING PLANE
       INTERSECTIONS ---
-
       %=========================================
       ==========================
-           fprintf('Finding osculating plane
       intersections...\n');
-
-           % Define the sample of z-points for the
       osculating planes
-           % (n_planes + 2 total, remove first and
       last, so n_planes points)
-           z_local_shockwave = linspace(0, width,
       n_planes + 2)';
-           z_local_shockwave =
       z_local_shockwave(2:end-1); % Keep only
       interior points
-
-           % Get y_bar values for the z sample
-           % (Was self.Get_Shockwave_Curve)
-           y_local_shockwave = zeros(n_planes, 1);
-           for i = 1:n_planes
-               z = z_local_shockwave(i);
-               if z <= width * X1
-                   y_local_shockwave(i) = 0;
-               else
-                   y_local_shockwave(i) =
       Interpolate_Shockwave(z);
-               end
-           end
-
-           % Find intersections with the upper
       surface
-           % (Was
       self.Find_Intersections_With_Upper_Surface)
-           local_intersections_us = zeros(n_planes,
       2); % [z, y_bar]
-           for i = 1:n_planes
-               z = z_local_shockwave(i);
-               if z <= X1 * width || X2 == 0
-                   local_intersections_us(i, 1) = z;
-                   local_intersections_us(i, 2) =
       Interpolate_Upper_Surface(z);
-               else
-                   % Find t, then derivative
-                   t = Find_t_Value(z, s_P0, s_P1,
       s_P2, s_P3, s_P4);
-                   [first_derivative, ~, ~] =
       First_Derivative(t, s_P0, s_P1, s_P2, s_P3,
       s_P4);
-
-                   % Find intersection
-                   y_s = y_local_shockwave(i);
-                   local_intersections_us(i, :) =
       Intersection_With_Upper_Surface(first_derivat
       ive, z, y_s, Interpolate_Upper_Surface,
       width);
-               end
-           end
-
       %=========================================
       ==========================
-           % --- 6. COMPUTE LEADING EDGE & CONE
       CENTERS ---
-
       %=========================================
       ==========================
-           fprintf('Computing leading edge and cone
       centers...\n');
-
-           % Initialize arrays (n_planes+2 points:
       tip, n_planes, base_tip)
-           leading_edge = zeros(n_planes + 2, 3); %
       [x, y, z] global coords
-           cone_centers = zeros(n_planes, 3);     %
       [x, y, z] global coords
-
-           % Tip is (0,0,0)
-           leading_edge(1, :) = [0, 0, 0];
-
-           % Base "tip" (point at the end of the
       shockwave)
-           leading_edge(end, :) = [length,
       Local_to_Global(X2 * height, height), width];
-
-           % (Was
       self.Compute_Leading_Edge_And_Cone_Centers)
-           for i = 1:n_planes
-               z = z_local_shockwave(i);
-               y_s_local = y_local_shockwave(i);
-               y_us_local =
       local_intersections_us(i, 2);
-
-               if useFlatBottom == true ||z <= X1 *
       width || X2 == 0
-                   % --- Flat Region ---
-                   cc_x = length - ((y_us_local -
       y_s_local) / tan(beta_rad));
-                   cc_y =
       Local_to_Global(y_us_local, height);
-                   cc_z = z;
-
-                   cone_centers(i, :) = [cc_x, cc_y,
       cc_z];
-                   leading_edge(i + 1, :) = [cc_x,
       cc_y, cc_z]; % LE is the cone center
-
-               else
-                   % --- Curved Region ---
-                   t = Find_t_Value(z, s_P0, s_P1,
       s_P2, s_P3, s_P4);
-                   [first_derivative, ~, ~] =
       First_Derivative(t, s_P0, s_P1, s_P2, s_P3,
       s_P4);
-                   radius =
       Calculate_Radius_Curvature(t, s_P0, s_P1,
       s_P2, s_P3, s_P4);
-
-                   theta_m = atan(first_derivative);
       % Angle of the normal
```

```
-
-                % Get cone center coordinates
    (global)
-                cc_x = length - radius /
    tan(beta_rad);
-                cc_y = Local_to_Global(y_s_local,
    height) + cos(theta_m) * radius;
-                cc_z = z - radius * sin(theta_m);
-                cone_centers(i, :) = [cc_x, cc_y,
    cc_z];
-
-                % Get leading edge coordinates
    (global)
-                x_S = length;
-                y_S_global =
    Local_to_Global(y_s_local, height);
-                z_S = z;
-                y_target_global =
    Local_to_Global(y_us_local, height);
-
-                leading_edge(i + 1, :) =
    Intersection_With_Freestream_Plane( ...
-                    cc_x, cc_y, cc_z, x_S,
    y_S_global, z_S, y_target_global);
-            end
-        end
-
-
-
    %=============================================
    ==========================
-        % --- 7. COMPUTE UPPER SURFACE GEOMETRY
    ---
-
    %=============================================
    ==========================
-        fprintf('Computing upper surface...\n');
-
-        % (Was self.Compute_Upper_Surface)
-        % We use a cell array for the streams, as
    in Python
-        upper_surface_streams = cell(n_planes +
    2, 1);
-
-        % Stream 1: Symmetry plane (z=0)
-        x_stream = linspace(0, length,
    n_streamwise)';
-        y_stream = zeros(n_streamwise, 1);
-        z_stream = zeros(n_streamwise, 1);
-        upper_surface_streams{1} = [x_stream,
    y_stream, z_stream];
-
-        % Streams 2 to n_planes+1 (the osculating
    planes)
-        for i = 1:n_planes
-            le_point = leading_edge(i + 1, :);
-            us_base_point_global = [length,
    Local_to_Global(local_intersections_us(i, 2),
    height), local_intersections_us(i, 1)];
-
-            x_stream = linspace(le_point(1),
    us_base_point_global(1), n_streamwise)';
-            y_stream = linspace(le_point(2),
    us_base_point_global(2), n_streamwise)';
-            z_stream = linspace(le_point(3),
    us_base_point_global(3), n_streamwise)';
-
-            upper_surface_streams{i + 1} =
    [x_stream, y_stream, z_stream];
-        end
-
-        % Stream n_planes+2: The final "tip"
    point
-        le_point = leading_edge(end, :);
-        upper_surface_streams{n_planes + 2} =
    [le_point; le_point]; % Just two points
-
-
-
    %=============================================
    ==========================
-        % --- 8. COMPUTE LOWER SURFACE GEOMETRY
    (STREAMLINE TRACING) ---
-
    %=============================================
    ==========================
-        fprintf('Computing lower surface
    (streamline tracing)...\n');
-
-        % Compute cone angle (the one on the
    cone, not the deflection)
-        cone_angle_deg = cone_angle(M_inf,
    beta_deg, gamma);
-        cone_angle_rad = cone_angle_deg * pi /
    180;
-
-        % Get the velocity-field interpolants
-        [Vrf, Vtf] = cone_field(M_inf,
    cone_angle_rad, beta_rad, gamma);
-
-        % --- Augment geometry arrays to include
    endpoints ---
-        % These arrays must match the
    leading_edge array (n_planes+2 entries)
-
-        y_local_shockwave_full = [0;
    y_local_shockwave; X2 * height];
-
-        z_local_shockwave_full = [0;
    z_local_shockwave; width];
-
-        % Need to add endpoints for cone_centers
    and intersections
-        cc_endpoint = [length, Local_to_Global(X2
    * height, height), width];
-        cone_centers_full = [zeros(1,3);
    cone_centers; cc_endpoint];
-
-        us_int_endpoint = [width, X2 * height];
-        local_intersections_us_full = [[0,
    height]; local_intersections_us;
    us_int_endpoint];
-
-        % --- Initialize storage ---
-        lower_surface_streams = cell(n_planes +
    2, 1);
-        max_step_size = delta_streamwise *
    length;
-
-        % --- Loop through all leading edge
    points ---
-        for i = 1:(n_planes + 2)
-            le_point = leading_edge(i, :);
-
-            if i == n_planes + 2
-                % --- Last "tip" point ---
-                lower_surface_streams{i} =
    [le_point; le_point];
-
-            elseif useFlatBottom == true
    ||z_local_shockwave_full(i) <= X1 * width ||
    X2 == 0
-                % --- Flat Region ---
-                % Simple trigonometry
-                bottom_surface_y = le_point(2) -
    tan(theta_rad) * (length - le_point(1));
-
-                x_stream = linspace(le_point(1),
    length, n_streamwise)';
-                y_stream = linspace(le_point(2),
    bottom_surface_y, n_streamwise)';
-                z_stream = repmat(le_point(3),
    n_streamwise, 1);
-
-                lower_surface_streams{i} =
    [x_stream, y_stream, z_stream];
-
-            else
-                % --- Curved Region (Osculating
    Cones) ---
-                % Get local geometry
-                cc = cone_centers_full(i, :);
```

```
-                y_s_local =
      y_local_shockwave_full(i);
-                y_us_local =
      local_intersections_us_full(i, 2);
-                z_local =
      z_local_shockwave_full(i);
-
-                % Calculate radii in the
      osculating plane
-                eta_le =
      Euclidean_Distance(local_intersections_us_ful
      l(i,1), Local_to_Global(y_us_local, height),
      cc(3), cc(2));
-                r_base =
      Euclidean_Distance(z_local,
      Local_to_Global(y_s_local, height), cc(3),
      cc(2));
-
-                % Get rotation angle for the
      plane
-                t = Find_t_Value(z_local, s_P0,
      s_P1, s_P2, s_P3, s_P4);
-                [m, ~, ~] = First_Derivative(t,
      s_P0, s_P1, s_P2, s_P3, s_P4);
-                alpha = atan(m);
-
-                % Initial conditions for ODE in
      2D cone-local coords [x', eta']
-                x_le = eta_le / tan(beta_rad);
-                x0 = [x_le; eta_le];
-
-                % Target "r" to stop integration
      (projected onto x-axis)
-                x_max = r_base / tan(beta_rad);
-
-                % Solve the streamline ODE
-                ode_options = odeset('Events',
      @(t,y) event_back(t,y,x_max), 'MaxStep',
      max_step_size);
-                [T, Y] = ode45(@(t,x)
      streamline_ode(t,x,Vrf,Vtf), [0, 1000], x0,
      ode_options);
-
-                % --- Transform 2D stream back to
      3D global coords ---
-                stream_2D = Y; % [x', eta']
-                stream_3D = zeros(size(stream_2D,
      1), 3);
-
-                % Rotate and position in 3D
-                stream_3D(:, 1) = stream_2D(:,
      1);                   % x-component
-                stream_3D(:, 2) = -stream_2D(:,
      2) * cos(alpha);     % y-component
-                stream_3D(:, 3) = stream_2D(:, 2)
      * sin(alpha);      % z-component
-
-                % Translate to global position
-                stream_3D(:, 1) = stream_3D(:, 1)
      + cc(1);
-                stream_3D(:, 2) = stream_3D(:, 2)
      + cc(2);
-                stream_3D(:, 3) = stream_3D(:, 3)
      + cc(3);
-
-                lower_surface_streams{i} =
      stream_3D;
-            end
-        end
-
-        fprintf('Streamline tracing
      complete.\n');
-
-
-
      %=========================================
      ==========================
-        % --- 9. PLOT & EXPORT (Equivalent of
      plotting_tools.py) ---
-
      %=========================================
      ==========================
-
-        % --- Plot 1: Base Plane ---
-        fprintf('Plotting base plane...\n');
-        figure;
-        hold on;
-        grid on;
-        axis equal;
-
-        % Get base points for lower surface
-        ls_base_points = zeros(n_planes + 2, 3);
-        for i = 1:(n_planes + 2)
-            ls_base_points(i, :) =
      lower_surface_streams{i}(end, :);
-        end
-
-        % --- Get base points for UPPER surface
      (in LOCAL y_bar coords) ---
-    % This data is stored in
      local_intersections_us_full from section 8
-    % We must re-create the array here.
-    us_int_endpoint = [width, X2 * height];
-    local_intersections_us_full = [[0, height];
      local_intersections_us; us_int_endpoint];
-
-    us_base_z = local_intersections_us_full(:,
      1);
-    us_base_y_bar =
      local_intersections_us_full(:, 2);
-
-    % Plot in y_bar (local) coordinates
-    plot(ls_base_points(:, 3), ls_base_points(:,
      2) + height, 'k-o', 'DisplayName', 'Lower
      Surface');
-    plot(us_base_z, us_base_y_bar, 'r-o',
      'DisplayName', 'Upper Surface');
-        % Plot shockwave
-        z_sw = z_local_shockwave_full;
-        y_sw = y_local_shockwave_full;
-        plot(z_sw, y_sw, 'g--', 'DisplayName',
      'Shockwave');
-
-        % Plot symmetry line
-        plot([0, 0], [0, height], 'b-',
      'DisplayName', 'Symmetry Plane');
-
-        title(sprintf('Base Plane
      [X1,X2,X3,X4]=[%.2f,%.2f,%.2f,%.2f]', X1, X2,
      X3, X4));
-        xlabel('z (Spanwise)');
-        ylabel('y_bar (Transverse, Local)');
-        legend('show');
-        hold off;
-
-        % --- Plot 2: Leading Edge Profile ---
-        fprintf('Plotting leading edge...\n');
-        figure;
-        hold on;
-        grid on;
-
-        plot(leading_edge(:, 1), leading_edge(:,
      3), 'b-o', 'DisplayName', 'Top View (x-z)');
-        plot(leading_edge(:, 1), leading_edge(:,
      2), 'r-o', 'DisplayName', 'Side View (x-y)');
-        xlabel('x (Streamwise)');
-        ylabel('y / z');
-        title('Leading Edge Profile');
-        legend('show');
-        hold off;
-
-        % --- EXPORT GEOMETRY TO CSV ---
-        % We cannot make CAD files directly, so
      we export point clouds.
-        % We must convert the cell arrays to a
      single matrix.
-
-        fprintf('Exporting geometry to CSV
      files...\n');
-
-        % Write Upper Surface
```

```
        % Combine all cells into one long matrix with an "ID" column
        % Col 1: Stream ID, Col 2: x, Col 3: y, Col 4: z
        us_export = [];
        for i = 1:numel(upper_surface_streams)
            stream = upper_surface_streams{i};
            id_col = ones(size(stream, 1), 1) * i;
            us_export = [us_export; [id_col, stream]];
        end
        writematrix(us_export, 'upper_surface.csv');

        % Write Lower Surface
        ls_export = [];
        for i = 1:numel(lower_surface_streams)
            stream = lower_surface_streams{i};
            id_col = ones(size(stream, 1), 1) * i;
            ls_export = [ls_export; [id_col, stream]];
        end
        writematrix(ls_export, 'lower_surface.csv');

        fprintf('Waverider generation complete.\n');
        fprintf('Saved: upper_surface.csv and lower_surface.csv\n');
    %================================================================
    % --- 10. ACCURATE L/D ESTIMATE (NEWTONIAN PANEL METHOD) ---
    %================================================================
    fprintf('--- L/D Estimate (Newtonian Panel Method) ---\n');

    % Call the panel method function
    [L_over_D, CL_total, CD_total] = calculate_panel_LD( ...
        upper_surface_streams, ...
        lower_surface_streams, ...
        aoa_deg, ...
        width, ...
        length);

    % --- 4. Display Results ---
    fprintf('Angle of Attack:   %.2f deg\n', aoa_deg);
    fprintf('Total CL:          %.4f\n', CL_total);
    fprintf('Total CD (Wave):   %.4f\n', CD_total);
    fprintf('Estimated L/D:     %.3f\n', L_over_D);
    fprintf('-------------------------------------\n');
    end % --- END OF MAIN FUNCTION ---
    %================================================================
    % --- LOCAL HELPER FUNCTIONS ---
    % (These are the MATLAB translations of the class methods and aux functions)
    %================================================================
    % --- Bézier Curve Functions ---

    function point = Bezier_Shockwave(t, P0, P1, P2, P3, P4)
        % 4th-order Bézier curve for the shockwave
        point = (1-t)^4 * P0 + ...
                4 * (1-t)^3 * t * P1 + ...
                6 * (1-t)^2 * t^2 * P2 + ...
                4 * (1-t) * t^3 * P3 + ...
                t^4 * P4;
    end

    function point = Bezier_Upper_Surface(t, P0, P1, P2, P3)
        % 3rd-order Bézier curve for the upper surface
        point = (1-t)^3 * P0 + ...
                3 * (1-t)^2 * t * P1 + ...
                3 * (1-t) * t^2 * P2 + ...
                t^3 * P3;
    end

    % --- Bézier Derivative Functions ---

    function [m, dzdt, dydt] = First_Derivative(t, P0, P1, P2, P3, P4)
        % First derivative of the 4th-order shockwave Bézier curve
        first_derivative_vec = 4 * (1-t)^3 * (P1 - P0) + ...
                               12 * (1-t)^2 * t * (P2 - P1) + ...
                               12 * (1-t) * t^2 * (P3 - P2) + ...
                               4 * t^3 * (P4 - P3);
        dzdt = first_derivative_vec(1);
        dydt = first_derivative_vec(2);
        m = dydt / dzdt; % slope
    end

    function [dzdt2, dydt2] = Second_Derivative(t, P0, P1, P2, P3, P4)
        % Second derivative of the 4th-order shockwave Bézier curve
        second_derivative_vec = 12 * (1-t)^2 * (P2 - 2*P1 + P0) + ...
                                24 * (1-t) * t * (P3 - 2*P2 + P1) + ...
                                12 * t^2 * (P4 - 2*P3 + P2);
        dzdt2 = second_derivative_vec(1);
        dydt2 = second_derivative_vec(2);
    end

    function radius = Calculate_Radius_Curvature(t, P0, P1, P2, P3, P4)
        % Calculates radius of curvature for the shockwave Bézier curve
        [~, dzdt, dydt] = First_Derivative(t, P0, P1, P2, P3, P4);
        [dzdt2, dydt2] = Second_Derivative(t, P0, P1, P2, P3, P4);

        numerator = (dzdt^2 + dydt^2)^(3/2);
        denominator = abs(dzdt * dydt2 - dydt * dzdt2);

        radius = numerator / denominator;
    end

    % --- Root-Finding & Intersection Functions ---

    function t = Find_t_Value(z, P0, P1, P2, P3, P4)
        % Finds the parameter 't' for a given 'z' on the shockwave curve

        % Pass the handle to the nested function to fzero
        t = fzero(@get_z_error, [0, 1]); % Find root in bracket [0, 1]

        % --- Nested Function ---
        % fzero requires a function that returns a scalar.
        % This nested function performs the call and the indexing in two steps.
        function scalar_val = get_z_error(t)
            % 1. Call the bezier function, which returns a [z,y] vector
```

```
            point_vec = Bezier_Shockwave(t, P0, P1, P2, P3, P4);

            % 2. Extract the z-component (the first element)
            z_val = point_vec(1);

            % 3. Return the scalar error for fzero
            scalar_val = z_val - z;
        end
        % --- End of Nested Function ---

    end

    function intersection = Intersection_With_Upper_Surface(first_derivative, z_s, y_s, Interpolate_Upper_Surface, width)
        % Finds intersection of the normal line with the upper surface curve
        m_normal = -1 / first_derivative;
        c_normal = y_s - m_normal * z_s;

        % Define the error function (distance between line and curve)
        fun = @(z) (m_normal * z + c_normal) - Interpolate_Upper_Surface(z);

        % Find the root (z-coordinate)
        z_int = fzero(fun, [0, width]);

        % Find the y-coordinate
        y_int = m_normal * z_int + c_normal;

        intersection = [z_int, y_int];
    end

    function intersection = Intersection_With_Freestream_Plane(x_C, y_C, z_C, x_S, y_S, z_S, y_target)
        % Finds intersection of line from cone center (C) to shock (S)
        % with the target y-plane (y_target)

        % k is the parametric distance
        k = (y_target - y_S) / (y_C - y_S);

        x_I = x_S + k * (x_C - x_S);
        y_I = y_target;
        z_I = z_S + k * (z_C - z_S);

        intersection = [x_I, y_I, z_I];
    end

    % --- Streamline ODE Functions ---

    function dxdt = streamline_ode(t, x, Vrf, Vtf)
        % ODE for tracing streamlines in the 2D osculating plane
        % x(1) = x' (streamwise-like)
        % x(2) = eta' (transverse-like)

        th = atan(x(2) / x(1)); % Angle in the 2D plane

        % Get velocities from the flowfield interpolants
        Vr_val = Vrf(th);
        Vt_val = Vtf(th);

        dxdt = zeros(2, 1);
        dxdt(1) = Vr_val * cos(th) - sin(th) * Vt_val;
        dxdt(2) = Vr_val * sin(th) + cos(th) * Vt_val;
    end

    function [value, isterminal, direction] = event_back(t, y, x_max)
        % Event function for streamline ODE
        % Stops when the x' component (y(1)) reaches the target x_max
        value = y(1) - x_max;
        isterminal = 1; % Stop integration
        direction = 0;  % All crossings
    end

    % --- Simple Math Helper Functions ---

    function y_global = Local_to_Global(y_local, height)
        % Converts y_bar (local) to y (global)
        y_global = y_local - height;
    end

    function dist = Euclidean_Distance(x1, y1, x2, y2)
        dist = sqrt((x2 - x1)^2 + (y2 - y1)^2);
    end

    function y = Equation_of_Line(z, m, c)
        y = m * z + c;
    end

    function c = cot(angle_rad)
        c = 1 / tan(angle_rad);
    End
```

- cone_angle.m

```
    function cone_angle_deg = cone_angle(Mach, shock_angle_deg, gamma)
    % Solves for the cone angle given a shock angle
    shock_angle_rad = shock_angle_deg * pi / 180;

    % Define the event options for the ODE solver
    options = odeset('Events', @(t,y) Vt0(t,y,gamma));

    % --- Oblique Shock Relations ---
    % Deflection angle
    d = atan(2.0 / tan(shock_angle_rad) * (Mach^2 * sin(shock_angle_rad)^2 - 1.0) / (Mach^2 * (gamma + cos(2 * shock_angle_rad)) + 2.0));

    % Post-shock Mach number
    Ma2 = 1.0 / sin(shock_angle_rad - d) * sqrt((1.0 + (gamma - 1.0) / 2.0 * Mach^2 * sin(shock_angle_rad)^2) / (gamma * Mach^2 * sin(shock_angle_rad)^2 - (gamma - 1.0) / 2.0));

    % Post-shock velocity
    V = 1.0 / sqrt(2.0 / ((gamma - 1.0) * Ma2^2) + 1.0);

    % Velocity components (initial conditions)
    Vr = V * cos(shock_angle_rad - d);
    Vt = -(V * sin(shock_angle_rad - d));
    xt = [Vr; Vt]; % Initial conditions as a column vector

    % --- Solve the ODE ---
    % Integrate from the shock angle down to 0, stopping at the Vt=0 event
    [T, Y, TE, YE, IE] = ode45(@(t,x) Taylor_Maccoll(t,x,gamma), [shock_angle_rad, 0.0], xt, options);

    if isempty(TE)
        error('No cone surface (Vt=0) found for the given shock angle.');
    end

    % Extract cone angle (the 'time' of the first event)
    cone_angle_rad = TE(1);

```

```
% Convert to degrees
cone_angle_deg = cone_angle_rad * 180 / pi;
End
```

- cone_field.m

```
function [Vrf, Vtf] = cone_field(Mach, theta_rad, beta_rad, gamma)
% Solves the conical flowfield and returns spline
% functions for the velocity components.

% --- Oblique Shock Relations ---
d = atan(2.0 / tan(beta_rad) * (Mach^2 * sin(beta_rad)^2 - 1.0) / (Mach^2 * (gamma + cos(2 * beta_rad)) + 2.0));
Ma2 = 1.0 / sin(beta_rad - d) * sqrt((1.0 + (gamma - 1.0) / 2.0 * Mach^2 * sin(beta_rad)^2) / (gamma * Mach^2 * sin(beta_rad)^2 - (gamma - 1.0) / 2.0));
V = 1.0 / sqrt(2.0 / ((gamma - 1.0) * Ma2^2) + 1.0);
Vr = V * cos(beta_rad - d);
Vt = -(V * sin(beta_rad - d));
xt = [Vr; Vt]; % Initial conditions

% --- Solve the ODE ---
% Integrate from the shock (beta_rad) to the cone (theta_rad)
[T, Y] = ode45(@(t,x) Taylor_Maccoll(t,x,gamma), [beta_rad, theta_rad], xt);

% --- Create Spline Functions ---
% ode45 returns T in descending order (beta_rad to theta_rad).
% Interpolants need ascending order, so we flip them.
T_asc = flip(T);
Vr_asc = flip(Y(:, 1));
Vtf_asc = flip(Y(:, 2));

% Create gridded interpolants (MATLAB's modern spline objects)
% Handle cases with too few points for a cubic spline
if numel(T_asc) < 4
    warning('Insufficient points for spline, using linear interpolation.');
    Vrf = griddedInterpolant(T_asc, Vr_asc, 'linear');
    Vtf = griddedInterpolant(T_asc, Vtf_asc, 'linear');
else
    Vrf = griddedInterpolant(T_asc, Vr_asc, 'spline');
    Vtf = griddedInterpolant(T_asc, Vtf_asc, 'spline');
end

End
```

- f_cone_root.m

```
function val = f_cone_root(shock_guess_rad, Mach, theta_rad, gamma)
% This is the function to find the root of.
% It calculates the cone angle for a given shock_guess_rad
% and returns the error (difference) from the target theta_rad.

% --- Oblique Shock Relations (based on guess) ---
x = shock_guess_rad; % 'x' is the shock angle guess
d = atan(2.0 / tan(x) * (Mach^2 * sin(x)^2 - 1.0) / (Mach^2 * (gamma + cos(2 * x)) + 2.0));
Ma2 = 1.0 / sin(x - d) * sqrt((1.0 + (gamma - 1.0) / 2.0 * Mach^2 * sin(x)^2) / (gamma * Mach^2 * sin(x)^2 - (gamma - 1.0) / 2.0));
V = 1.0 / sqrt(2.0 / ((gamma - 1.0) * Ma2^2) + 1.0);
Vr = V * cos(x - d);
Vt = -(V * sin(x - d));
xt = [Vr; Vt];

% --- Solve the ODE ---
options = odeset('Events', @(t,y) Vt0(t,y,gamma));
[T, Y, TE, YE, IE] = ode45(@(t,y) Taylor_Maccoll(t,y,gamma), [x, 0.0], xt, options);

if isempty(TE)
    % No event found, this is a bad guess. Return a large error.
    val = 1e6;
else
    % Event found. TE(1) is the resulting cone angle.
    % Return the error.
    val = TE(1) - theta_rad;
end
End
```

- shock_angle.m

```
function beta_deg = shock_angle(Mach, theta_deg, gamma)
% Solves for the shock angle given a cone angle
theta_rad = theta_deg * pi / 180;

% Create an anonymous function to pass parameters to fsolve
% fsolve will try to make this function equal zero
fun = @(shock_guess_rad) f_cone_root(shock_guess_rad, Mach, theta_rad, gamma);

% Initial guess for beta (shock angle) is just theta
% We also set options to turn off the solver's display
options = optimset('Display','off');
beta_rad = fsolve(fun, theta_rad, options);

beta_deg = beta_rad * 180 / pi;
End
```

- Taylor_Maccoll.m

```
function dxdt = Taylor_Maccoll(t, x, gamma)
% Solves the Taylor-Maccoll ODE which describes conical flow
% t = angle (theta)
% x = state vector [Vr; Vt]
%     x(1) = Vr (radial velocity)
%     x(2) = Vt (tangential velocity)
% gamma = ratio of specific heats

% Calculate A constant
A = (gamma - 1.0) / 2.0 * (1.0 - x(1)^2 - x(2)^2);

dxdt = zeros(2, 1); % Initialize as a column vector

% d(Vr)/dt
dxdt(1) = x(2);

% d(Vt)/dt
dxdt(2) = (x(2) * x(1) * x(2) - A * (2.0 * x(1) + x(2) / tan(t))) / (A - x(2) * x(2));

End
```

- Vt0.m

```
function [value, isterminal, direction] = Vt0(t, y, gamma)
% Event function for ode45 to find where Vt = 0
% y(2) is the tangential velocity (Vt)
```

```
    value = y(2);      % Stop when this value is
    zero
    isterminal = 1;  % Stop the integration
    (True)
    direction = 0;   % Find all zero crossings
    End
```

- calculate_panel_LD.m

```
    function [L_over_D, CL_total, CD_total] =
    calculate_panel_LD(upper_surface_streams,
    lower_surface_streams, aoa_deg, width,
    length)
    %
    ============================================
    ============================
    %  calculate_panel_LD.m
    %  Estimates L/D using a Newtonian Panel
    Method.
    %  This function calculates wave drag
    (pressure) only. It neglects
    %  viscous drag (friction).
    %
    ============================================
    ============================

        % --- 1. Setup ---
        aoa_rad = aoa_deg * pi / 180;

        % Freestream velocity vector (normalized)
        V_inf = [cos(aoa_rad); sin(aoa_rad); 0];

        % Reference Area (total span * length)
        S_ref = (2 * width) * length;

        % --- 2. Process Surfaces ---
        % Process the upper and lower surfaces to
    get total forces
        [Fx_upper, Fy_upper] =
    process_surface(upper_surface_streams,
    aoa_rad, V_inf, true);
        [Fx_lower, Fy_lower] =
    process_surface(lower_surface_streams,
    aoa_rad, V_inf, false);

        % --- 3. Sum Forces ---
        % Total forces in the BODY axis (x, y)
        Total_Fx = Fx_upper + Fx_lower;
        Total_Fy = Fy_upper + Fy_lower;

        % --- 4. Rotate Forces to WIND Axis (Lift
    & Drag) ---
        % L = F_y*cos(a) - F_x*sin(a)
        % D = F_y*sin(a) + F_x*cos(a)
        Total_L_force = Total_Fy * cos(aoa_rad) -
    Total_Fx * sin(aoa_rad);
        Total_D_force = Total_Fy * sin(aoa_rad) +
    Total_Fx * cos(aoa_rad);

        % --- 5. Non-Dimensionalize ---
        % The forces are currently (Force /
    q_inf). We divide by S_ref.
        CL_total = Total_L_force / S_ref;
        CD_total = Total_D_force / S_ref;

        L_over_D = Total_L_force / Total_D_force;

    end

    %
    ============================================
    ============================
    %  Helper Function: process_surface
    %
    ============================================
    ============================
    function [Fx_surface, Fy_surface] =
    process_surface(streams, aoa_rad, V_inf,
    isUpper)

        Fx_surface = 0;
        Fy_surface = 0;

        num_streams = numel(streams);

        % Check if streams are empty or have data
        if num_streams < 2
            return;
        end

        num_points_on_stream = size(streams{1},
    1);

        % Loop through all streamlines and points
    to create panels
        for i = 1:(num_streams - 1)

            % Get the two streamlines that form
    the panel strip
            stream_A = streams{i};
            stream_B = streams{i+1};

            % Ensure streams have the same number
    of points
            if size(stream_A, 1) ~=
    size(stream_B, 1)
                % This can happen at the tip,
    skip this panel
                continue;
            end

            for j = 1:(num_points_on_stream - 1)
                % --- a. Define the 4 corners of
    the panel ---
                P1 = stream_A(j,   :);
                P2 = stream_B(j,   :);
                P3 = stream_B(j+1, :);
                P4 = stream_A(j+1, :);

                % --- b. Get Panel Geometry (Area
    and Normal Vector) ---
                % Use cross product of diagonals
                v1 = P3 - P1;
                v2 = P4 - P2;
                n_vec = cross(v1, v2);

                % Ensure normal vector points OUT
    of the surface
                % Upper surface normal should
    have y > 0
                % Lower surface normal should
    have y < 0
                if isUpper && n_vec(2) < 0
                    n_vec = -n_vec;
                elseif ~isUpper && n_vec(2) > 0
                    n_vec = -n_vec;
                end

                n_norm = norm(n_vec);

                % If area is zero, skip
                if n_norm < 1e-12
                    continue;
                end

                Area = 0.5 * n_norm;
                n_hat = n_vec / n_norm; %
    Normalized normal vector

                % --- c. Apply Newtonian Theory
    ---
                Cp = 0;
                dot_prod = dot(V_inf, n_hat);

                % Only windward surfaces
    (dot_prod > 0) generate pressure
                if dot_prod > 0
                    Cp = 2 * (dot_prod)^2;
                end

                % --- d. Calculate Force in Body
    Axis ---
```

```
-                % Pressure force vector F = -Cp *
    Area * n_hat
-                % (Pressure pushes *into* the
    surface, opposite the normal)
-                Force_vec = -Cp * Area * n_hat;
-
-                % Sum the x and y components
-                Fx_surface = Fx_surface +
    Force_vec(1); % Force in x-direction
-                Fy_surface = Fy_surface +
    Force_vec(2); % Force in y-direction
-            end
-        end
-    End
```

- FLIRConversion.m

```
-    % 1. Load your CSV file
-    data = readmatrix('Rec-0100_400.csv');
-
-    % 2. Generate the Heatmap Image
-    figure;
-    imagesc(data); % 'imagesc' scales the colors
    automatically based on temp range
-
-    % 3. Style it to look like a FLIR image
-    colormap(jet);
-
-    colorbar;      % Adds the temperature scale
    bar on the right
-    axis image;    % Ensures the image isn't
    stretched or distorted
-    axis off;      % Removes the X and Y axis
    numbers
-
-    title('Restored Thermal Image');
-
-    % 4. Save the result
-    saveas(gcf, 'restored_image.jpg');
```

B. ANSYS Notes

All ANSYS Fluent CFD simulations were done with the same refined mesh that reduced residuals to the negative third power at convergence. They were also all done on a body cut at the symmetry plane, meaning the lift and drag force values would be doubled in reality. The flow conditions were M 6 freestream speed, 229.7 K inlet temperature, gauge temperature of 30800 Pa, 5% turbulent intensity, turbulent viscosity ratio of 10, and a backflow total temperature of 1884 K. The air material was modified with an ideal gas density, NASA-9-piecewise-polynomial specific heat, kinetic theory thermal conductivity, sutherland viscosity, and constant molecular weight, characteristic length, and energy parameter. In terms of models, the energy equation was turned on and the viscous model was SST k-omega with viscous heating and compressibility effects turned on.

C. ANSYS Fluent Results at Various Angles of Attack

0 degree Angle of Attack										
Continuity	x-velocity	y-velocity	z-velocity	energy	k	omega	Lift (N)	Drag (N)	Lift Coefficient	Drag Coefficient
1.54e-3	1.16e-8	1.15e-8	1.06e-8	1.54e-6	1.25e-3	3.10e-3	496.6	486.4	0.125	0.123
4 degree Angle of Attack										
Continuity	x-velocity	y-velocity	z-velocity	energy	k	omega	Lift (N)	Drag (N)	Lift Coefficient	Drag Coefficient
1.032e-4	6.767e-10	8.56e-10	6.75e-10	1.385e-7	5.01e-4	1.399e-3	1897	692.3	0.494	0.185
6 degree Angle of Attack										

Continuity	x-velocity	y-velocity	z-velocity	energy	k	omega	Lift (N)	Drag (N)	Lift Coefficient	Drag Coefficient
1.715e-3	1.659e-8	1.492e-8	1.29e-8	2.043e-6	4.389e-4	1.11e-3	2873	926.5	0.726	0.234
7 degree Angle of Attack										
Continuity	x-velocity	y-velocity	z-velocity	energy	k	omega	Lift (N)	Drag (N)	Lift Coefficient	Drag Coefficient
1.849e-3	1.67e-8	2.133e-8	1.50e-8	2.111e-6	4.281e-4	1.258e-3	3179.9	1333	0.804	0.2624
10 degree Angle of Attack										
Continuity	x-velocity	y-velocity	z-velocity	energy	k	omega	Lift (N)	Drag (N)	Lift Coefficient	Drag Coefficient
4.99e-3	3.631e-8	7.05e-8	2.939e-8	5.534e-6	4.705e-4	8.122e-4	4646	1572	1.17	.397
11 degree Angle of Attack										
Continuity	x-velocity	y-velocity	z-velocity	energy	k	omega	Lift (N)	Drag (N)	Lift Coefficient	Drag Coefficient
3.271e-3	3.359e-8	5.69e-8	2.47e-8	4.999e-6	6.62e-4	1.15e-3	5174	1798.9	1.309	.4551

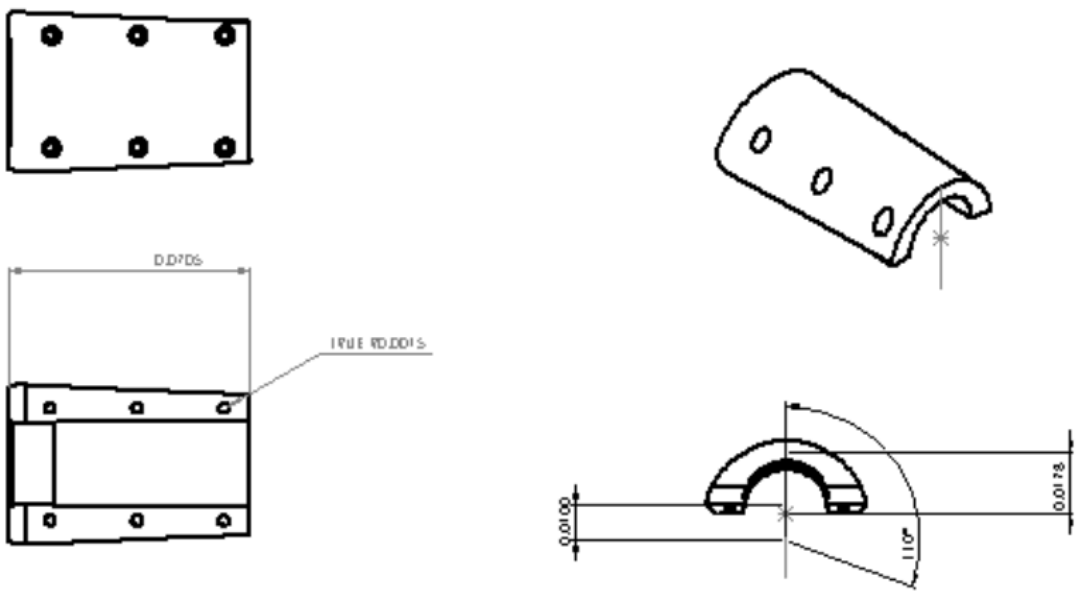
BIBLIOGRAPHY

[1] Becker, J. V., 1964, "Studies of High Lift/Drag Ratio Hypersonic Configurations," *Proceedings of the 4th Congress of the International Council of the Aeronautical Sciences (ICAS)*, Paris, France, AIAA Paper 64-551. https://doi.org/10.2514/6.1964-551

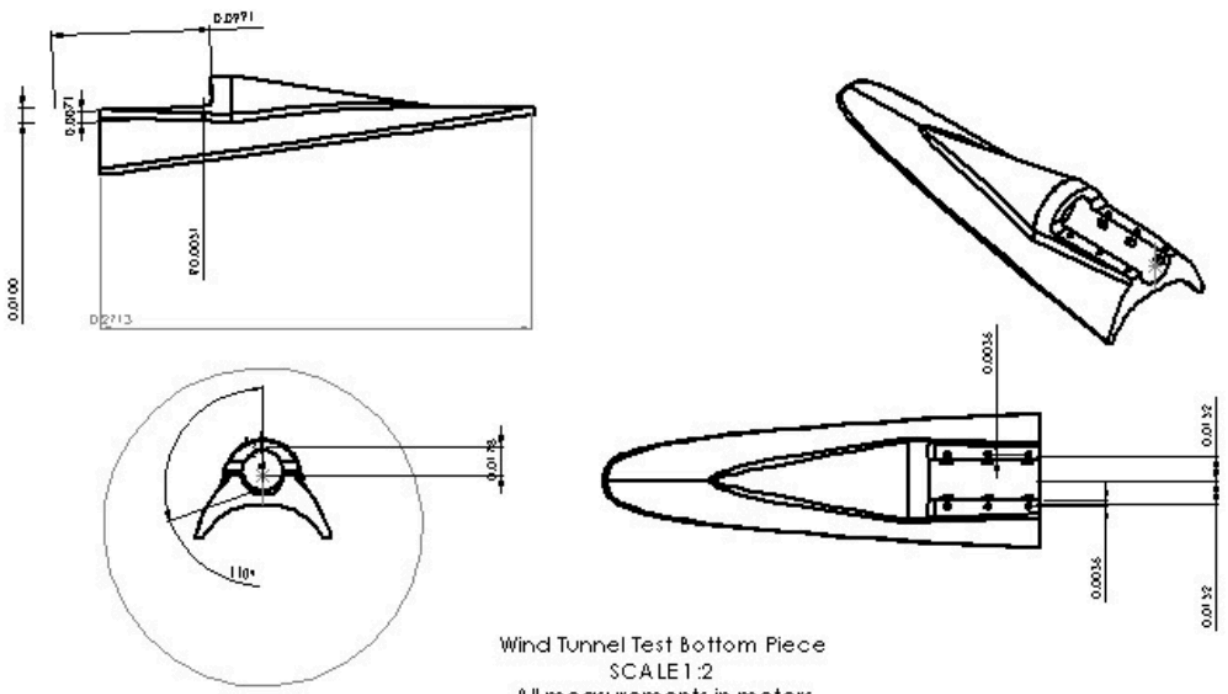
[2] Zhen-tao Zhao, Wei Huang, Li Yan, and Yan-guang Yang, 2020, "An Overview of Research on Wide-Speed Range Waverider Configuration," *Progress in Aerospace Sciences*, 113, p. 100606. https://doi.org/10.1016/j.paerosci.2020.100606
[3] Jin, Z., Yu, Z., Meng, F., Zhang, W., Cui, J., He, X., Lei, Y., and Musa, O., 2024, "Parametric Design Method and Lift/Drag Characteristics Analysis for a Wide-Range, Wing-Morphing Glide Vehicle," *Aerospace*, 11(4), p. 257. https://doi.org/10.3390/aerospace11040257
[4] Pan, X., Yue, H., Liu, S., Yang, F., Lu, Y., and Chen, G., 2023, "Research on the Performance of Slender Aircraft with Flare-Stabilized-Skirt," *Aerospace*, **10**(10), p. 844. https://doi.org/10.3390/aerospace10100844
[5] Gloria, H. R., 1956, "An Investigation of the Lift, Drag, and Static-Stability Characteristics of a Triangular-Wing Airplane Configuration at Mach Numbers from 3.00 to 6.28," *NACA Research Memorandum*, NACA RM A56H27, National Advisory Committee for Aeronautics, Washington, D.C. https://ntrs.nasa.gov/citations/19930089475
[6] Aerospaceweb.org, n.d., "Waveriders," *Aerospaceweb.org*. https://aerospaceweb.org/design/waverider/main.shtml. Accessed December 9, 2025.
[7] Shuai-Qi Guo, Wen Liu, Chen-An Zhang, Yang Liu, and Fa-Min Wang, 2024, "Aerodynamic Optimization of Hypersonic Blunted Waveriders Based on Symbolic Regression," *Aerospace Science and Technology*, 144, p. 108801. https://doi.org/10.1016/j.ast.2023.108801

CAD Drawings

Wind Tunnel Articles

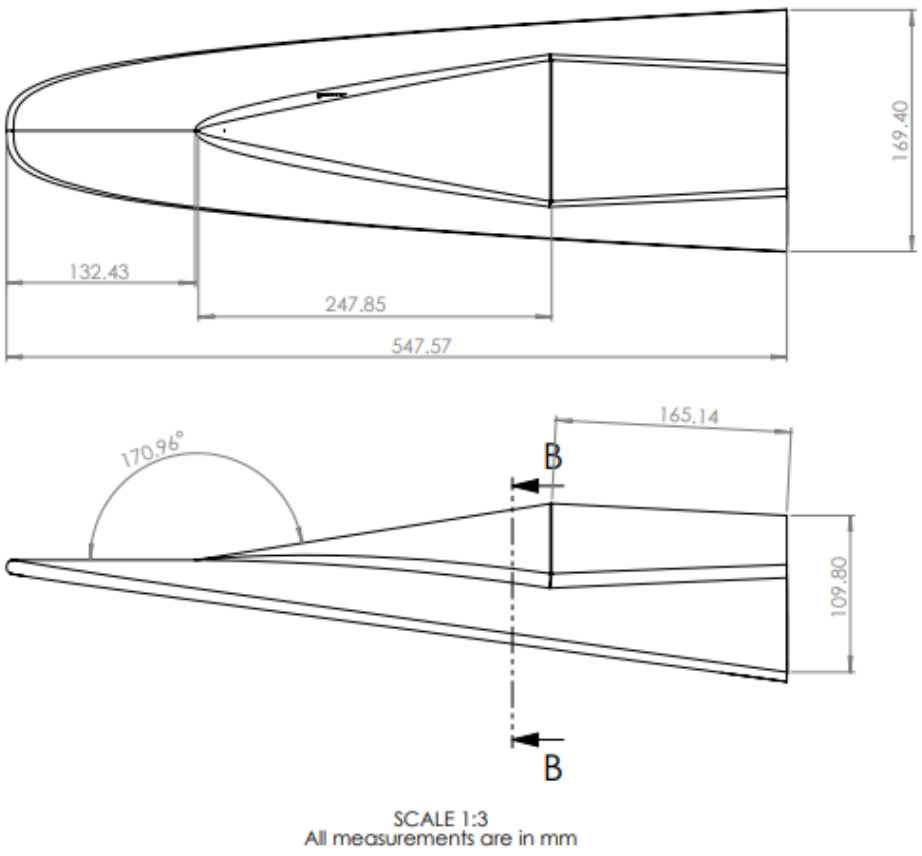


Wind Tunnel Test Article Top
SCALE 1:1
All measurements in meters

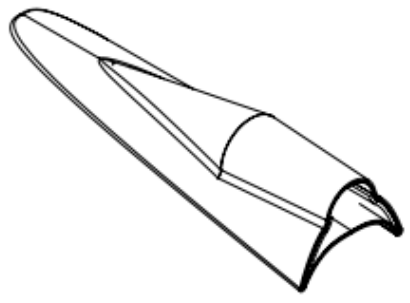


Wind Tunnel Test Bottom Piece
SCALE 1:2
All measurements in meters

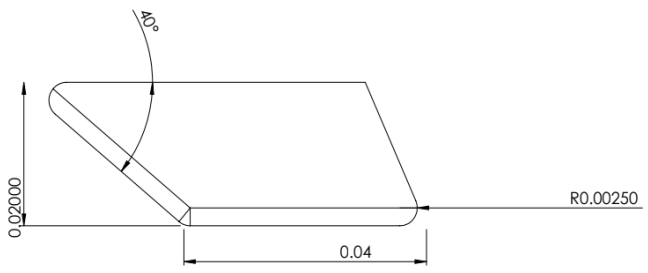
Gun Test Articles

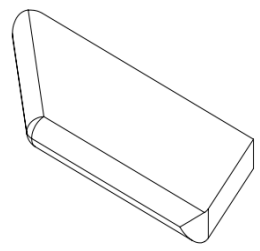


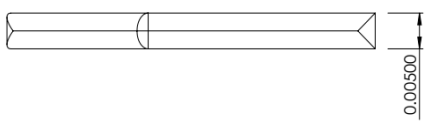
SCALE 1:3
All measurements are in mm

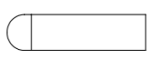


SECTION B-B
SCALE 1 : 2

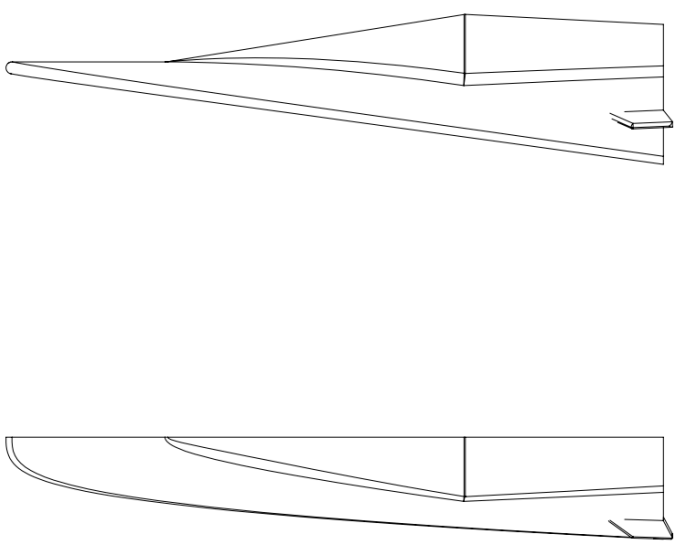








Fin Design
SCALE 2:1
All measurments in meters



Gun test Model with Fin Half Body
SCALE 1:3
All measurements in meters